ToyGPT 강의 노트

"처음부터 만드는 언어 모델 — Pre-training → Fine-tuning → Chat"

목차

1. 전체 파이프라인 개요
2. 토크나이저 — 텍스트를 숫자로
3. 데이터셋 — 언어 모델의 학습 방식
4. 모델 구조 — ToyGPT 해부
5. 사전 학습 — 01_pretrain.py
6. 파인튜닝 — 02_finetune.py
7. 추론과 대화 — 03_chat.py
8. 한국어 파이프라인 — 영어와의 차이점
9. GPU 지원 — 자동 스케일업

1. 전체 파이프라인 개요

이 프로젝트는 ChatGPT, Claude 같은 대형 언어 모델(LLM)이 어떻게 만들어지는지를 **세 단계**로 보여주는 교육용 예제입니다.

```
┌──────────────────────────────────────────────────────────────────┐
│                      LLM 학습 파이프라인                          │
├──────────────────────┬──────────────────────┬──────────────────────┤
│  Stage 1             │  Stage 2             │  Stage 3             │
│  사전 학습           │  파인튜닝            │  대화                │
│  Pre-training        │  Fine-tuning         │  Inference / Chat    │
├──────────────────────┴──────────────────────┴──────────────────────┤
│ [영어 파이프라인]                                                    │
│  01_pretrain.py   →  02_finetune.py   →  03_chat.py  (선택: 1)      │
│  Shakespeare          영어 Q&A 25쌍       영어 모델 대화             │
│  ~2-3분 (CPU)         ~30초 (CPU)                                   │
├──────────────────────────────────────────────────────────────────┤
│ [한국어 파이프라인]                                                  │
│  01_pretrain_ko.py → 02_finetune_ko.py → 03_chat.py  (선택: 2)      │
│  한국 전래동화        한국어 Q&A 25쌍      한국어 모델 대화           │
│  ~5분 (CPU)           ~1분 (CPU)                                    │
│  ~1분 (GPU)           ~10초 (GPU)                                   │
└──────────────────────────────────────────────────────────────────┘
```

파일 역할 요약

파일	역할
`tokenizer.py`	텍스트 ↔ 정수 변환 (문자 단위, 영어/한국어 공통)

파일	역할
`model.py`	GPT 트랜스포머 구조 정의 (영어/한국어 공통)
`data.py`	학습 데이터셋 로드 및 가공 (영어/한국어 공통)
`01_pretrain.py`	**[영어]** Shakespeare 사전 학습
`02_finetune.py`	**[영어]** 영어 Q&A 파인튜닝
`01_pretrain_ko.py`	**[한국어]** 전래동화 사전 학습 + GPU 자동 스케일업
`02_finetune_ko.py`	**[한국어]** 한국어 Q&A 파인튜닝
`03_chat.py`	대화 인터페이스 — 실행 시 영어/한국어 모델 선택

모델 스펙 비교

항목	영어 모델 (CPU)	한국어 모델 CPU	한국어 모델 GPU
사전 학습 데이터	Shakespeare ~1.1M	전래동화 ~43K×8	동일
vocab_size	~70 (영문자)	~470 (한글 음절)	동일
d_model	64	64	**128**
n_layers	2	2	**4**
n_heads	4	4	**8**
d_ff	256	256	**512**
총 파라미터	**~112K**	**~340K**	**~4.2M**
context_len	128	128	128
파인튜닝 데이터	영어 Q&A 25쌍	한국어 Q&A 25쌍	동일

---

2. 토크나이저 — 텍스트를 숫자로

왜 토크나이저가 필요한가?

신경망은 숫자만 처리할 수 있습니다. "Hello" → `[7, 4, 11, 11, 14]` 처럼 텍스트를 정수로 변환해야 합니다.

왜 문자(Character) 단위인가?

방식	vocab 크기	코드 복잡도	교육 적합성
문자 단위	~70개	매우 낮음	★★★ 최적
BPE (GPT-4 방식)	50,000개	높음	★
단어 단위	수만 개	중간	★★

문자 단위라면 vocab 전체를 화면에 한 번에 출력해서 볼 수 있습니다.

vocab 구성 원리 (`tokenizer.py:23-26`)

```
# 텍스트에서 고유 문자를 추출하고 정렬
chars = sorted(set(text))

# 각 문자에 정수 ID를 부여
self.vocab     = {ch: i for i, ch in enumerate(chars)}   # 'A'→0, 'B'→1, ...
self.inv_vocab = {i: ch for ch, i in self.vocab.items()} # 0→'A', 1→'B', ...
```

> `sorted()`를 사용하는 이유: 같은 텍스트라면 항상 같은 vocab이 만들어지도록 **재현성** 보장.

encode / decode (`tokenizer.py:40-57`)

```
def encode(self, text: str) -> list:
    """문자열 → 정수 리스트"""
    return [self.vocab[ch] for ch in text if ch in self.vocab]

def decode(self, ids: list) -> str:
    """정수 리스트 → 문자열"""
    return ''.join(self.inv_vocab.get(i, '') for i in ids)
```

동작 예시:

```
encode("Hello") → [7, 4, 11, 11, 14]
decode([7, 4, 11, 11, 14]) → "Hello"
```

체크포인트 저장/로드 (`tokenizer.py:59-80`)

토크나이저는 학습 후에도 같은 vocab을 사용해야 합니다. `01_pretrain.py`에서 `tokenizer.json`으로 저장하고, 이후 모든 스크립트에서 로드합니다.

```
tokenizer.save("checkpoints/tokenizer.json")   # 01_pretrain.py에서
tokenizer = CharTokenizer.load("checkpoints/tokenizer.json")  # 이후 스크립트
```

> **핵심 요약: 토크나이저는 텍스트와 정수 사이의 번역기. vocab은 학습 코퍼스에서 자동 생성된다.**

---

3. 데이터셋 — 언어 모델의 학습 방식

"다음 토큰 예측" 목표

언어 모델의 학습 목표는 단 하나입니다:

> **이전 문자들이 주어졌을 때, 다음 문자를 예측하라.**

이것이 GPT, Claude, Gemini 등 모든 현대 LLM의 기본 학습 방식입니다.

```
입력(x): "To be, or not to b"
정답(y): "o be, or not to be"
         ↑ x를 한 칸 오른쪽으로 shift
```

레이블이 필요 없습니다. 텍스트 자체가 입력이자 정답입니다. 이를 **자기지도학습(Self-supervised Learning)** 이라고 합니다.

Sliding Window 방식 (`data.py:164-181`)

```python
class TextDataset(Dataset):
    def __getitem__(self, idx: int):
        x = self.data[idx     : idx + self.context_len]     # 입력
        y = self.data[idx + 1 : idx + self.context_len + 1] # 정답 (한 칸 shift)
        return x, y
```

시각적 예시 (context_len=4):

```
전체 토큰: [10, 20, 30, 40, 50, 60, 70]

인덱스 0: x=[10, 20, 30, 40]  y=[20, 30, 40, 50]
인덱스 1: x=[20, 30, 40, 50]  y=[30, 40, 50, 60]
인덱스 2: x=[30, 40, 50, 60]  y=[40, 50, 60, 70]
```

모든 위치가 동시에 입력이기도 하고 정답이기도 합니다. **데이터 효율이 매우 높습니다.**

셰익스피어 데이터 (`data.py:57-83`)

```python
def get_shakespeare(cache_path: str = "shakespeare.txt") -> str:
    """
    Tiny Shakespeare 텍스트 (~1.1MB) 다운로드.
    캐시가 있으면 파일에서 로드, 없으면 인터넷에서 다운로드.
    다운로드 실패 시 내장 백업 텍스트 사용.
    """
```

- 크기: 약 1,115,394 문자
- 출처: Andrej Karpathy의 char-rnn 프로젝트 (공개 도메인)
- ML 분야의 "Hello World" 데이터셋

> **핵심 요약: 언어 모델은 텍스트를 한 칸씩 밀어 자기 자신을 정답으로 삼아 학습한다. 별도의 라벨이 필요 없다.**

---

4. 모델 구조 — ToyGPT 해부

전체 아키텍처

```
입력: 토큰 ID [t0, t1, t2, ..., t_{T-1}]
        │
        ▼
┌──────────────────────────────┐
│   Token Embedding            │  각 토큰 ID → 64차원 벡터
│   +                          │
│   Position Embedding         │  각 위치(0~127) → 64차원 벡터
└──────────────────────────────┘
        │ (두 벡터를 더함)
        ▼
┌──────────────────────────────┐
│   TransformerBlock × 2       │
│  ┌────────────────────────┐  │
│  │  LayerNorm             │  │
│  │  CausalSelfAttention   │  │  "어떤 이전 토큰에 집중할까?"
│  │  + Residual            │  │
│  ├────────────────────────┤  │
│  │  LayerNorm             │  │
│  │  FeedForward (MLP)     │  │  "각 위치에서 정보를 변환"
│  │  + Residual            │  │
│  └────────────────────────┘  │
└──────────────────────────────┘
        │
        ▼
┌──────────────────────────────┐
│   LayerNorm (최종)           │
└──────────────────────────────┘
        │
        ▼
┌──────────────────────────────┐
│   LM Head (Linear)           │  64차원 → 70차원 (vocab_size)
│   (가중치 = Token Embedding) │  ← Weight Tying!
└──────────────────────────────┘
        │
        ▼
출력: logits [각 위치에서 다음 토큰의 점수]
      → softmax → 확률 분포 → 샘플링
```

핵심 개념 1: Causal Mask (인과적 마스크)

문제: 어텐션(Attention)은 기본적으로 모든 위치를 서로 볼 수 있습니다. 그런데 학습 시 미래 토큰을 보면 "정답을 미리 보고 푸는" 것과 같습니다.

해결: 미래 위치를 `-inf`로 마스킹 → softmax 후 0이 됨 → 완전히 무시.

```
# model.py:58-61
# 하삼각행렬: 현재 위치 이하만 1, 이후는 0
mask = torch.tril(torch.ones(context_len, context_len))
self.register_buffer('mask', mask)
```

마스크 시각화 (context_len=5):

```
        t0  t1  t2  t3  t4  ← 참조할 위치 (Key)
t0  [  1,   0,   0,   0,   0  ]   ← t0은 자신만 봄
t1  [  1,   1,   0,   0,   0  ]   ← t1은 t0, t1만 봄
t2  [  1,   1,   1,   0,   0  ]   ← t2는 t0~t2만 봄
t3  [  1,   1,   1,   1,   0  ]
t4  [  1,   1,   1,   1,   1  ]   ← t4는 모두 봄
     ↑ 현재 위치 (Query)
```

```
# model.py:76-80
scores = scores.masked_fill(
    self.mask[:T, :T] == 0,
    float('-inf')   # softmax(-inf) = 0
)
```

> **핵심 요약: Causal Mask는 언어 모델이 "치팅(미래 보기)"을 못 하도록 막는 삼각형 마스크다.**

핵심 개념 2: Scaled Dot-Product Attention

어텐션의 수학적 공식:

```
Attention(Q, K, V) = softmax( Q × Kᵀ / √d_k ) × V
```

기호	의미	비유
Q (Query)	"내가 무엇을 찾고 있나?"	검색어
K (Key)	"나는 어떤 정보를 갖고 있나?"	색인
V (Value)	"실제로 전달할 정보"	내용
√d_k	스케일링	점수가 너무 크면 softmax가 날카로워짐 → 그래디언트 소실

코드로 보기 (`model.py:67-82`):

```
# Q, K, V 계산
qkv = self.qkv_proj(x)                # 한 번에 3배 크기로 선형 변환
Q, K, V = qkv.split(C, dim=-1)        # 3등분
```

```
# Scaled Dot-Product
scale  = math.sqrt(self.d_head)
scores = (Q @ K.transpose(-2, -1)) / scale   # (B, H, T, T)

# Causal Masking + Softmax
scores       = scores.masked_fill(self.mask[:T, :T] == 0, float('-inf'))
attn_weights = F.softmax(scores, dim=-1)

# V와 가중합
out = attn_weights @ V
```

> **핵심 요약: 어텐션은 "어떤 이전 토큰에 얼마나 집중할지" 점수를 계산해 가중 평균을 내는 연산이다.**

핵심 개념 3: Residual Connection + Pre-LayerNorm

Residual Connection (잔차 연결):

```
# model.py:148-149
x = x + self.attn(self.ln1(x))   # 어텐션 출력 + 원본 더하기
x = x + self.ff(self.ln2(x))     # FFN 출력 + 원본 더하기
```

왜 필요한가?

```
잔차 연결 없음:  입력 → Block1 → Block2 → ... → 출력
                       그래디언트가 역전파되면서 점점 사라짐 (소실)

잔차 연결 있음:  입력 ──────────────────────────→ 출력
                    ↘ Block1 ↗ ↘ Block2 ↗
                       그래디언트가 단축 경로로 직접 흐름 (소실 방지)
```

Pre-LayerNorm (GPT-2 방식):

```
Pre-LN (이 코드):  x → LN → Attention → + x
Post-LN (원논문):  x → Attention → + x → LN
```

Pre-LN이 학습이 더 안정적입니다.

> **핵심 요약: Residual Connection은 깊은 신경망에서 그래디언트가 사라지는 문제를 해결한다.**

핵심 개념 4: Weight Tying (가중치 공유)

```
# model.py:208
self.lm_head.weight = self.token_embedding.weight
```

- **Token Embedding**: 정수 ID → 64차원 벡터 (입력에서 사용)
- **LM Head**: 64차원 → 70차원 (출력에서 사용)

두 행렬의 크기는 동일합니다: `(vocab_size × d_model) = (70 × 64)`

같은 가중치를 공유하면:

- 파라미터 수 감소 (70 × 64 = 4,480개 절약)
- **같은 토큰이 입력과 출력에서 유사한 표현을 가져야 한다는 직관** 반영
- GPT-2, GPT-3도 동일하게 사용

> **핵심 요약: Weight Tying은 입력 임베딩과 출력 행렬을 공유해 파라미터를 절약하고 일관성을 높인다.**

파라미터 수 계산

레이어	크기	파라미터
Token Embedding	70 × 64	4,480
Position Embedding	128 × 64	8,192
TransformerBlock × 2		
QKV Projection	64 × 192 × 2	24,576
Out Projection	64 × 64 × 2	8,192
FFN Linear 1	64 × 256 × 2	32,768
FFN Linear 2	256 × 64 × 2	32,768
LayerNorm × 4	64 × 2 × 4	512
LN Final	64 × 2	128
LM Head	(공유됨)	0
합계		**~112,000**

---

5. 사전 학습 — 01_pretrain.py

개요

```
셰익스피어 텍스트 (~1.1M 문자)
        ↓ 토크나이저로 인코딩
정수 시퀀스 (~1.1M 토큰)
        ↓ Sliding Window
학습 샘플 (x, y) 쌍
```

```
        ↓ 반복 학습 (5 epoch)
사전 학습된 모델 가중치
```

핵심 학습 루프 (`01_pretrain.py:156-177`)

```
for epoch in range(1, MAX_EPOCHS + 1):
    for x, y in train_loader:

        # 1. 그래디언트 초기화
        optimizer.zero_grad()

        # 2. 순전파 (Forward Pass)
        logits, loss = model(x, targets=y)

        # 3. 역전파 (Backward Pass) — 그래디언트 계산
        loss.backward()

        # 4. Gradient Clipping — 폭발적 그래디언트 방지
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

        # 5. 가중치 업데이트
        optimizer.step()
```

Cross-Entropy Loss 란?

모델이 예측한 확률 분포와 실제 정답 사이의 거리를 측정합니다.

```
정답 토큰: 'e' (ID=14)
모델 예측: 'e' 확률 = 0.8  → Loss 작음 (잘 예측함)
모델 예측: 'e' 확률 = 0.01 → Loss 큼  (틀리게 예측함)

Loss = -log(정답 토큰의 예측 확률)
```

epoch가 진행될수록 Loss가 감소 → 모델이 셰익스피어 문체를 학습 중.

Gradient Clipping 이 필요한 이유

트랜스포머는 간혹 그래디언트가 폭발적으로 커지는 경우가 있습니다. `max_norm=1.0`으로 전체 그래디언트의 크기를 1.0 이하로 제한합니다.

```
# 그래디언트 노름이 1.0을 초과하면 비율적으로 축소
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

하이퍼파라미터 (`01_pretrain.py:37-49`)

```
CONTEXT_LEN   = 128    # 128개 문자를 보고 129번째 예측
BATCH_SIZE    = 64     # 한 번에 64개 샘플 처리
LEARNING_RATE = 3e-4   # AdamW 학습률 (3 × 10⁻⁴)
MAX_EPOCHS    = 5      # 전체 데이터를 5번 반복
```

수업 중 실연 포인트

각 epoch 종료 후 모델이 생성하는 텍스트를 출력합니다:

```
--- Epoch 1 생성 샘플 ---
ROMEO: roh tthe thhhhe  hh tte...   ← 아직 노이즈에 가까움

--- Epoch 3 생성 샘플 ---
ROMEO: the the the and the...       ← 영어 단어가 나타나기 시작

--- Epoch 5 생성 샘플 ---
ROMEO: I will not be a man,
  That shall be so good...          ← 셰익스피어 문체와 유사해짐
```

학생들이 눈으로 학습 진행을 확인할 수 있는 가장 인상적인 순간입니다.

영어 vs 한국어 사전 학습 데이터 비교

	영어 (`01_pretrain.py`)	**한국어 (`01_pretrain_ko.py`)**
데이터 출처	Tiny Shakespeare (URL 다운로드)	전래동화 텍스트 (스크립트 내 하드코딩)
크기	~1.1M 문자	~43K 문자 × 8 반복 = ~344K
vocab_size	~70 (ASCII 영문자)	~470 (한글 음절 + 구두점)
이유	외부 URL 한 줄로 로드 가능	한국어 공개 URL 접근이 불안정 → 내장

한국어 vocab이 ~470개인 이유:

```
한글 음절 = 자음 19개 × 모음 21개 × 받침 28개 = 최대 11,172개
→ 그러나 전래동화에 실제로 등장하는 음절은 ~450개
→ 거기에 공백, 구두점, Q:, A:, 숫자 등 ~20개 추가
→ 총 ~470개
```

> **핵심 요약: 사전 학습은 "다음 문자 예측"을 수백만 번 반복하면서 언어의 패턴을 학습한다. 라벨이 필요 없다.**

---

6. 파인튜닝 — 02_finetune.py

사전 학습 vs 파인튜닝 비교

	사전 학습	파인튜닝
데이터	대량 (1.1M 문자)	소량 (25 Q&A쌍, ~5K 문자)
학습률	높음 (`3e-4`)	낮음 (`1e-4`)
Epoch	적음 (5)	많음 (30)
목적	언어 일반 패턴 학습	특정 형식/태스크 적응
출발점	랜덤 가중치	사전 학습된 가중치

Q&A 데이터 형식 (`02_finetune.py:107-117`)

```
def build_finetune_text(qa_pairs: list) -> str:
    parts = []
    for question, answer in qa_pairs:
        parts.append(f"\nQ: {question}\nA: {answer}\n")
    return "".join(parts)
```

학습 텍스트 예시:

```
Q: Who wrote Romeo and Juliet?
A: William Shakespeare wrote Romeo and Juliet around 1594.

Q: What is the famous line from Hamlet?
A: To be, or not to be, that is the question.

Q: Who is Juliet's lover?
A: Juliet's lover is Romeo, a young man from the Montague family.
...
```

모델은 이 패턴을 반복 학습하여 `Q:` 다음에는 `A:`로 답하는 방식을 익힙니다.

왜 학습률을 낮추는가?

```
사전 학습에서 습득한 지식 (영어 문법, 셰익스피어 문체)
    → 파인튜닝 시 너무 빠르게 덮어쓰면 잊어버림 ("파국적 망각")
    → 낮은 LR로 기존 지식을 보존하면서 새 형식만 학습
```

Before / After 비교 (강의 하이라이트)

동일한 질문 `Q: Who is Romeo?`에 대한 응답:

```
[ Before 파인튜닝 ]
Q: Who is Romeo?
```

```
A: roh the and the the man, and the...   ← Q&A 형식을 모름, 셰익스피어 잡음

[ After 파인튜닝 ]
Q: Who is Romeo?
A: Romeo is a young man from the Montague family who falls in love with Juliet.
```

같은 가중치 구조, 25개 예시만 추가했는데 응답 방식이 완전히 달라집니다. 이것이 파인튜닝의 힘입니다.

> **핵심 요약: 파인튜닝은 사전 학습된 모델의 지식을 보존하면서, 소량의 데이터로 특정 형식이나 태스크에 적응시킨다.**

---

7. 추론과 대화 — 03_chat.py

자기회귀 생성 (Autoregressive Generation)

이것이 ChatGPT, Claude가 실제로 동작하는 방식입니다.

핵심: 토큰 하나 생성 = forward pass 1회

```python
# 03_chat.py:96-135 (강의 핵심 코드)

context = torch.tensor([seed_ids], dtype=torch.long)

for _ in range(max_new_tokens):

    # 1. context_len 초과 시 잘라냄
    context_crop = context[:, -model.context_len:]

    # 2. Forward Pass — 현재 컨텍스트로 다음 토큰 예측
    logits, _ = model(context_crop)
    next_logits = logits[0, -1, :]      # 마지막 위치 logits만 사용

    # 3. Temperature 스케일링
    next_logits = next_logits / temperature

    # 4. Top-k 필터링
    top_vals, _ = torch.topk(next_logits, k=40)
    threshold   = top_vals[-1]
    next_logits = next_logits.masked_fill(next_logits < threshold, float('-inf'))

    # 5. 확률 분포에서 샘플링
    probs      = torch.softmax(next_logits, dim=-1)
    next_token = torch.multinomial(probs, num_samples=1)

    # 6. 실시간 스트리밍 출력 ← 이 한 줄이 ChatGPT 스트리밍의 원리
    char = tokenizer.decode([next_token.item()])
    print(char, end='', flush=True)

    # 7. 새 토큰을 컨텍스트에 추가
    context = torch.cat([context, next_token.unsqueeze(0)], dim=1)
```

이 루프의 의미:

```
100 토큰 생성 = forward pass 100회!
이것이 왜 GPU가 필요한지, 왜 모델이 클수록 느린지의 이유입니다.
```

Temperature — 창의성 조절

Temperature는 확률 분포의 "날카로움"을 조절합니다:

```
next_logits = next_logits / temperature  # temperature로 나눔
```

Temperature	효과	예시
0.1 (낮음)	항상 가장 확률 높은 토큰 선택	"the the the the..." (반복)
0.8 (기본)	적당한 다양성	자연스러운 문장
2.0 (높음)	무작위에 가까움	"xQ9 wandering purple..." (혼란)

시각화:

```
Temperature = 0.1 (날카로운 분포)    Temperature = 2.0 (평평한 분포)
  확률
  |▉                                  |▉▉
  | ▉                                 |  ▉▉
  |  ▉▉                               |    ▉▉▉▉
  |    ▉▉▉▉▉▉▉▉                       |        ▉▉▉▉▉▉▉▉▉▉▉▉
  └──────────────→ 토큰               └──────────────────→ 토큰
    "the"가 거의 확실                   여러 토큰이 비슷한 확률
```

Top-k 필터링

확률이 낮은 토큰들을 완전히 제거합니다 (상위 k개만 유지):

```
# 상위 40개 토큰의 logit만 남기고 나머지는 -inf
top_vals, _ = torch.topk(next_logits, k=40)
threshold   = top_vals[-1]
next_logits = next_logits.masked_fill(next_logits < threshold, float('-inf'))
```

효과: 완전히 관계없는 문자(예: 숫자 중간에 갑자기 특수문자)가 나올 확률 제거

스트리밍 출력의 원리

ChatGPT가 답변을 한 글자씩 보여주는 이유:

```
# 이 한 줄이 전부입니다
print(char, end='', flush=True)
```

- `end=''`: 줄바꿈 없이 바로 이어서 출력
- `flush=True`: 버퍼에 쌓지 않고 즉시 터미널에 출력

모델은 실제로 한 번에 전체 답변을 생성하지 않습니다. **토큰 하나를 생성할 때마다 화면에 즉시 표시하는 것입니다.**

채팅 명령어

명령어	기능
`/temp 0.5`	temperature를 0.5로 변경
`/pretrain`	파인튜닝 전 모델로 전환 (비교용)
`/finetune`	파인튜닝 후 모델로 전환
`/reset`	대화 초기화
`/quit`	종료

`/pretrain`과 `/finetune`을 번갈아 사용하면 같은 질문에 두 모델의 차이를 실시간으로 비교할 수 있습니다.

영어/한국어 모델 선택 (`03_chat.py`)

실행 시 두 파이프라인 중 하나를 선택합니다:

```
# 03_chat.py 실행 시 출력
사용할 모델을 선택하세요:
  [1] 영어 모델 (Shakespeare 사전 학습)
  [2] 한국어 모델 (전래동화 사전 학습)
선택 (1 또는 2, 기본값 1):
```

내부적으로 다른 체크포인트 경로를 로드합니다:

```
CONFIGS = {
    "en": {
        "pretrain" : "checkpoints/pretrain_final.pt",
        "finetune" : "checkpoints/finetune_final.pt",
        "tokenizer": "checkpoints/tokenizer.json",
    },
    "ko": {
        "pretrain" : "checkpoints/pretrain_ko_final.pt",
        "finetune" : "checkpoints/finetune_ko_final.pt",
        "tokenizer": "checkpoints/tokenizer_ko.json",
    },
}
```

> **핵심 요약: LLM은 토큰 하나씩 생성하며, 매번 forward pass를 1회 실행한다. 스트리밍 출력은 각 토큰이 생성될 때마다 즉시 화면에 표시하는 것이다.**

---

8. 한국어 파이프라인 — 영어와의 차이점

왜 영어 모델에 한국어를 파인튜닝할 수 없는가?

이것이 한국어 **별도 파이프라인**이 필요한 이유입니다.

```
영어 사전 학습 → vocab = {' ', '!', 'A', 'B', ... 'z'} 약 70개
                         ↑ 한글 문자가 전혀 없음!

02_finetune.py에서 한국어 Q&A를 학습하려 하면:
    finetune_text = ''.join(ch for ch in text if ch in tokenizer.vocab)
                            ↑ 한글 문자는 모두 필터링됨 → 빈 문자열

결과: 한국어 Q&A 데이터가 학습에 전혀 사용되지 않음
```

해결책: 한국어 텍스트로 처음부터 사전 학습 → 한국어 vocab 생성 → 한국어 파인튜닝

영어 vs 한국어 파이프라인 핵심 차이 3가지

차이 1: 사전 학습 데이터

```
[영어]  Shakespeare (URL에서 다운로드)
        약 1.1M 문자, vocab ~70개 (ASCII 문자)

[한국어] 전래동화 (스크립트 내 하드코딩)
         흥부놀부, 콩쥐팥쥐, 토끼와 거북이, 금도끼 은도끼,
         단군신화, 심청이, 해님달님, 속담 등
         약 43K 문자 × 8 반복 = ~344K, vocab ~470개 (한글 음절)
```

한국어를 하드코딩한 이유:

- 한국어 공개 대용량 텍스트의 안정적 URL 확보가 어려움
- 전래동화는 파인튜닝 Q&A 주제(흥부놀부, 단군 등)와 연결되어 교육 효과 극대화

차이 2: vocab_size에 따른 모델 크기 변화

vocab_size가 크면 embedding table이 커집니다:

```
영어:  Token Embedding = 70  × 64  =   4,480개 파라미터
한국어: Token Embedding = 470 × 64  =  30,080개 파라미터  (6.7배)
```

```
한국어 CPU 모델 총 파라미터: ~340K
한국어 GPU 모델 총 파라미터: ~4.2M (d_model=128, n_layers=4)
```

차이 3: 파인튜닝 Q&A 주제

```
# 02_finetune_ko.py — 한국어 Q&A 예시
QA_PAIRS_KO = [
    ("흥부와 놀부 이야기에서 흥부는 어떤 사람인가요?",
     "흥부는 마음씨가 착하고 부지런한 사람입니다."),

    ("한글을 만든 사람은 누구인가요?",
     "한글은 조선 시대 세종대왕이 1443년에 창제하였습니다."),

    ("가는 말이 고와야 오는 말이 곱다는 속담의 뜻은?",
     "남에게 좋게 대해야 나도 좋은 대우를 받는다는 뜻입니다."),
    # ... 총 25쌍
]
```

주제 분류: 전래동화(8쌍), 한국 역사/문화(9쌍), 속담/지혜(4쌍), 자연/계절(4쌍)

한국어 Before / After 비교

`Q: 흥부와 놀부에서 흥부는 어떤 사람인가요?`

```
[ Before 파인튜닝 ]
A: 옛날 옛적에 흥부와 놀부라는 형제가 살았습니다 형 놀부는...
   (Q&A 형식을 모르고 전래동화 본문을 그대로 이어씀)

[ After 파인튜닝 ]
A: 흥부는 마음씨가 착하고 부지런한 사람입니다. 가난하게 살면서도
   착한 마음을 잃지 않았습니다.
   (Q&A 형식에 맞게 간결하게 답변)
```

> **핵심 요약: 언어가 다르면 vocab이 달라지므로 반드시 해당 언어로 사전 학습을 해야 한다. 영어 모델에 한국어를 파인튜닝하는 것은 불가능하다.**

---

9. GPU 지원 — 자동 스케일업

GPU 자동 감지 (`01_pretrain_ko.py`)

```
def get_device_and_config():
    """GPU 유무에 따라 디바이스와 모델 크기를 결정합니다."""
    if torch.cuda.is_available():
        device = torch.device("cuda")
        gpu_name = torch.cuda.get_device_name(0)
```

```
        gpu_mem  = torch.cuda.get_device_properties(0).total_memory / 1e9
        print(f"GPU 감지: {gpu_name}")
        print(f"GPU 메모리: {gpu_mem:.1f} GB")
        # GPU는 더 큰 모델 사용 → 더 좋은 품질
        model_cfg = dict(d_model=128, n_layers=4, n_heads=8, d_ff=512)
        batch_size = 128
    else:
        device = torch.device("cpu")
        print("CPU 모드 (GPU 없음)")
        model_cfg = dict(d_model=64, n_layers=2, n_heads=4, d_ff=256)
        batch_size = 64
    return device, model_cfg, batch_size
```

CPU vs GPU 모델 크기 자동 조정

항목	CPU 모드	GPU 모드	이유
d_model	64	128	GPU는 병렬 연산이 강력
n_layers	2	4	더 깊은 모델 = 더 풍부한 표현력
n_heads	4	8	더 많은 어텐션 헤드
d_ff	256	512	더 넓은 FFN
batch_size	64	128	GPU 메모리 활용
총 파라미터	**~340K**	**~4.2M**	12배 차이

pin_memory — GPU 전송 최적화

```
# GPU가 있을 때만 pin_memory=True
use_pin = device.type == 'cuda'
train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True,
                          pin_memory=use_pin)
```

pin_memory란?

```
일반 메모리 (pageable):        고정 메모리 (pinned):
CPU RAM → [복사] → GPU VRAM   CPU RAM ──→ GPU VRAM
          ↑ 동기 복사                ↑ 비동기 DMA 전송
          학습 루프이 기다림          다음 배치 준비와 병렬 실행
```

- GPU 학습 시 데이터 로딩 병목을 줄여 약 10~20% 속도 향상
- CPU 모드에서는 효과 없으므로 자동으로 비활성화

GPU 메모리 모니터링

```
# 매 epoch마다 GPU 메모리 사용량 출력
if device.type == 'cuda':
    mem = torch.cuda.memory_allocated() / 1e6
    print(f"  GPU 메모리 사용: {mem:.1f} MB")
```

예상 학습 시간

파이프라인	CPU	GPU (RTX 3060)	GPU (A100)
영어 사전 학습 (5 epoch)	~2-3분	~20초	~5초
영어 파인튜닝 (30 epoch)	~30초	~5초	~2초
한국어 사전 학습 (5 epoch)	~5분	~1분	~15초
한국어 파인튜닝 (40 epoch)	~1분	~10초	~3초

GPU 사용 여부 확인

```
import torch
print(torch.cuda.is_available())        # True/False
print(torch.cuda.get_device_name(0))    # GPU 이름
print(torch.cuda.device_count())        # GPU 개수
```

> **핵심 요약: GPU가 있으면 모델 크기를 자동으로 키워 더 좋은 품질의 모델을 학습한다. pin_memory로 CPU→GPU 데이터 전송을 최적화한다.**

전체 핵심 개념 요약

#	개념	한 줄 요약
1	문자 단위 토크나이저	텍스트를 정수로 변환. 영어 ~70개, 한국어 ~470개
2	다음 토큰 예측	언어 모델의 유일한 학습 목표. 라벨 불필요
3	Causal Mask	미래를 보지 못하게 하는 삼각형 마스크
4	Scaled Attention	Q/K/V로 "어디에 집중할지" 계산. √d_k로 안정화
5	Residual Connection	그래디언트 소실 방지. 입력을 출력에 더함
6	Weight Tying	임베딩과 LM Head 가중치 공유
7	사전 학습	대량 데이터로 언어 일반 패턴 습득. 언어별 별도 필요
8	파인튜닝	소량 데이터로 특정 형식/태스크 적응. 낮은 LR
9	Autoregressive	토큰 하나씩 생성. N 토큰 = N 번의 forward pass
10	Temperature	낮을수록 확정적, 높을수록 창의적

#	개념	한 줄 요약
11	Top-k 필터링	확률 낮은 토큰 제거로 출력 품질 향상
12	스트리밍	생성 즉시 출력. ChatGPT 동작 원리와 동일
13	언어별 별도 파이프라인	vocab이 다르면 사전 학습도 달라야 함. 영어→한국어 파인튜닝 불가
14	GPU 자동 스케일업	GPU 감지 시 d_model·n_layers 자동 확장. pin_memory로 전송 최적화

실행 방법

```
# 패키지 설치 (공통)
uv pip install -r requirements.txt
```

영어 파이프라인

```
python 01_pretrain.py       # ~2-3분 (CPU)
                            # → checkpoints/pretrain_final.pt
                            # → checkpoints/tokenizer.json
                            # → loss_curve.png

python 02_finetune.py       # ~30초 (CPU)
                            # → Before/After 비교 출력
                            # → checkpoints/finetune_final.pt

python 03_chat.py           # → 실행 후 '1' 선택
```

한국어 파이프라인

```
python 01_pretrain_ko.py    # ~5분 (CPU) / ~1분 (GPU)
                            # → checkpoints/pretrain_ko_final.pt
                            # → checkpoints/tokenizer_ko.json
                            # → loss_curve_ko.png

python 02_finetune_ko.py    # ~1분 (CPU) / ~10초 (GPU)
                            # → Before/After 한국어 비교 출력
                            # → checkpoints/finetune_ko_final.pt

python 03_chat.py           # → 실행 후 '2' 선택
```

GPU 확인

```
python -c "import torch; print(torch.cuda.is_available(),
torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'No GPU')"
```